

Static Part Average Testing (SPAT)

vs

Dynamic Part Average Testing (DPAT)

Why Dynamic PAT Can Become Less Reliable in Semiconductor Backend Assembly & Test

Viren Patel
Decoded with Viren — Technical White Paper

February 2, 2026

Disclaimer: This document is based on public engineering principles and author-developed models. All scenarios, datasets, and results are illustrative and/or simulated and do not represent any specific company, customer, product, test limits, or proprietary manufacturing process. Views are the author's own.

Abstract

Part Average Testing (PAT) is widely used in semiconductor backend test to screen abnormal parts using parametric outlier detection. Two commonly deployed methodologies are **Static Part Average Testing (SPAT)**, where limits are derived from large historical production datasets, and **Dynamic Part Average Testing (DPAT)**, where limits are updated adaptively using recent test windows (e.g., rolling lots or time windows). While DPAT can offer faster reaction time to localized shifts, this paper argues that DPAT can become **less reliable and less effective in backend assembly and test environments** due to stacked variability sources (materials, firmware configurations, equipment/toolchain drift, measurement noise, and routing/rework variation). Conversely, SPAT provides improved screening stability and scalability in high-mix backend conditions by anchoring acceptance limits to large-volume population behavior. DPAT becomes most effective only when backend conditions are **ultra-stable and fully traceable**.

Contents

1	Introduction	3
1.1	Key thesis	3
2	Definitions and Terminology	3
2.1	Part Average Testing (PAT)	3
2.2	Static Part Average Testing (SPAT)	3
2.3	Dynamic Part Average Testing (DPAT)	3
3	Variance Stack in Backend Assembly & Test	4
3.1	Primary variability contributors	4
4	Why DPAT Becomes Less Reliable in Backend	4
4.1	DPAT failure mechanisms	5
4.2	Risk: false rejects and false accepts	5

5	Why SPAT is More Robust in Backend	5
6	SPAT vs DPAT Comparison	6
7	When DPAT Works Best	6
8	Recommended Hybrid PAT Framework	6
9	Case Study: Representative MCP Backend Scenario (Illustrative)	6
9.1	Background	6
9.2	Observed issue	7
9.3	Investigation findings	7
9.4	Root cause	7
9.5	SPAT outcome	7
10	Simulation Study: Rolling Baseline Fragility in DPAT	7
10.1	Objective	7
10.2	Method	8
10.3	Evaluation metrics	8
10.4	Expected outcome	8
11	Interactive Simulation Extension (Portfolio)	8
12	Conclusion	8

1. Introduction

Semiconductor backend manufacturing (Assembly & Test) is a high-variance environment compared to wafer fab. Product performance and measured parametric results are influenced not only by silicon behavior, but also by: materials and packaging variations, firmware and calibration routines, equipment differences, thermal handling, test program changes, contactor/socket wear, measurement system repeatability (GR&R), and routing/rework history.

Because PAT is intended to screen abnormal parts and provide early detection of distribution-level changes, the choice between SPAT and DPAT significantly impacts yield stability, false reject risk, and debug cycle time.

1.1. Key thesis

DPAT only works reliably when the system is stable and traceable. Backend environments are inherently multi-variable; therefore DPAT can become noisy and misleading. In such conditions, SPAT is generally more robust as a primary screening gate.

2. Definitions and Terminology

2.1. Part Average Testing (PAT)

PAT is an outlier-screening method applied to parametric test results to identify units that deviate abnormally from the population.

2.2. Static Part Average Testing (SPAT)

SPAT uses large historical populations (high N) to define stable baseline distributions and acceptance limits. Typical implementations use:

- Mean and standard deviation (μ, σ) with robust variants
- Median and MAD-based sigma
- Percentile-based limits (P1/P99, etc.)
- Long-window control charts (EWMA/CUSUM optional)

Interpretation: SPAT acts as a **global population baseline**.

2.3. Dynamic Part Average Testing (DPAT)

DPAT calculates PAT limits using recent production results (rolling window) and can be contextualized by: tester ID, site, temperature, configuration, firmware revision, or routing. Typical implementations use:

- Last N units or last M lots
- Rolling time windows (e.g., last 24 hours / 7 days)
- Per-tool / per-site baselines
- Adaptive baseline updates

Interpretation: DPAT acts as a **local rolling baseline**.

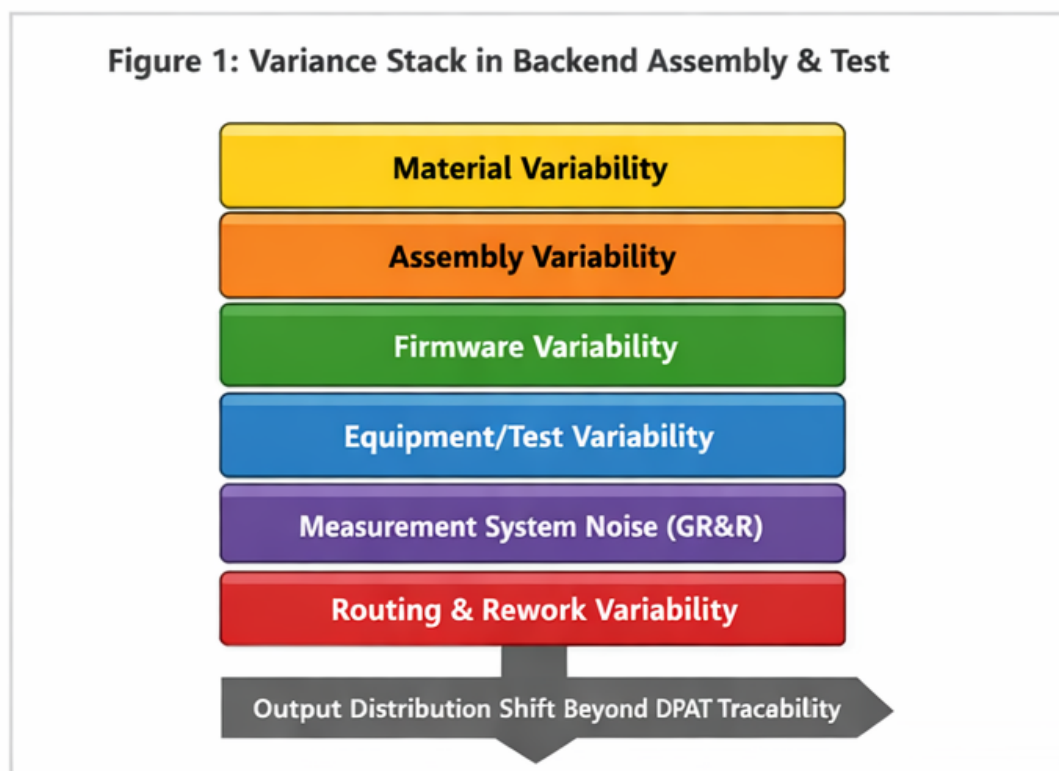


Figure 1: Variance stack in backend assembly & test (illustrative).

3. Variance Stack in Backend Assembly & Test

Backend behavior is a stacked-variance system. Even if the silicon is stable, the measured parametric distribution can shift due to multiple interacting factors.

3.1. Primary variability contributors

1. **Material variability:** suppliers, substrate lots, mold compound, solder paste, adhesives.
2. **Assembly variability:** bonding, curing, warpage, voiding, contamination micro-variation.
3. **Firmware variability:** FW revisions, training/calibration algorithms, trim methods.
4. **Equipment/test variability:** tester type, socket aging, handler thermal drift, loadboard differences.
5. **Measurement system noise:** GR&R limitations in parametric measurement repeatability.
6. **Routing/rework variability:** insertion history, thermal exposure differences, rework loops.

4. Why DPAT Becomes Less Reliable in Backend

DPAT's rolling baseline assumes that the most recent population represents a valid equivalence class. However, backend systems often violate equivalence assumptions due to incomplete traceability and non-linear interactions between factors.

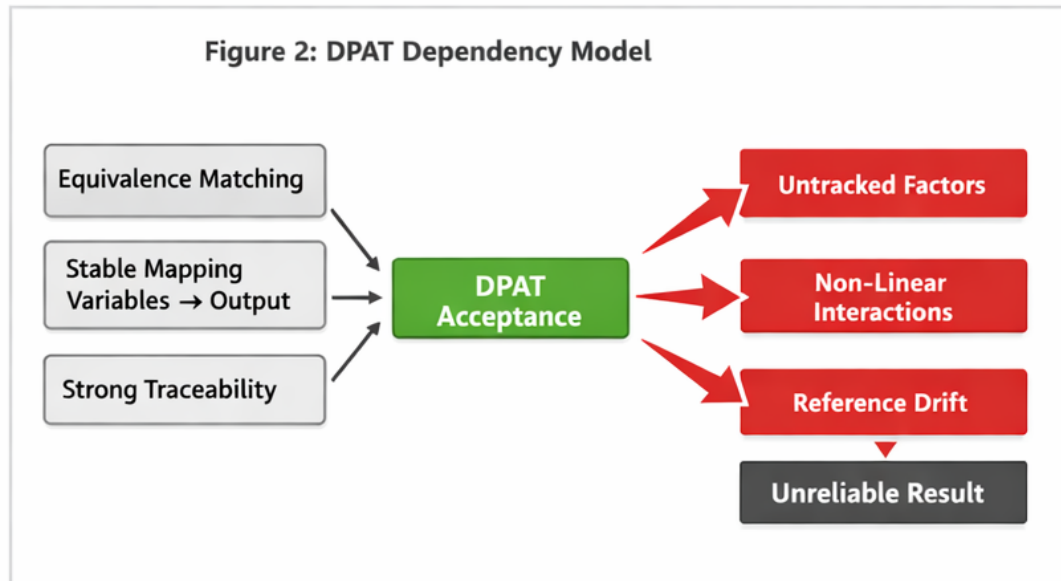


Figure 2: DPAT dependency model: failure modes caused by untracked factors and baseline drift.

4.1. DPAT failure mechanisms

- **Untracked drivers:** Many dominant backend drivers are not fully represented in traceability data.
- **Non-linear interactions:** e.g., FW × socket aging × temperature compensation.
- **Reference drift:** rolling baselines drift with equipment wear and calibration cycles.

4.2. Risk: false rejects and false accepts

In high variance environments DPAT can increase both:

- **False rejects:** healthy parts rejected because baseline shifted due to noise.
- **False accepts:** weak parts accepted because local baseline degraded.

5. Why SPAT is More Robust in Backend

SPAT is anchored to large population behavior and is therefore less sensitive to short-term noise, temporary tester shifts, or contactor/socket wear trends. SPAT enables stable screening even across:

- multi-supplier BOM mixes,
- frequent routing changes,
- equipment heterogeneity,
- firmware updates across product life cycle.

Figure 3: SPAT vs. DPAT Comparison

Statistical PAT (SPAT)	Dynamic PAT (DPAT)
Backend Readiness	Low
Stability Dependency	High
False Reject Risk	High
False Accept Risk	High
Scalability	Moderate
Debugging Ease	Easier

Figure 3: High-level comparison: SPAT vs DPAT for backend readiness and reliability.

6. SPAT vs DPAT Comparison

7. When DPAT Works Best

DPAT becomes highly effective when backend conditions approximate wafer fab stability:

- minimal BOM and material variance,
- frozen firmware/training/calibration methods,
- equipment homogeneity with strong matching practices,
- high traceability maturity including tool capability state,
- controlled routing and minimized rework loops.

8. Recommended Hybrid PAT Framework

A practical deployment in backend uses both:

- **SPAT as primary acceptance gate** (robust global screening),
- **DPAT as targeted diagnostic layer** (fast local anomaly detection).

9. Case Study: Representative MCP Backend Scenario (Illustrative)

9.1. Background

This scenario represents a typical memory MCP backend manufacturing line where multiple variability sources co-exist (BOM mix, FW versions, tester diversity, and contactor wear).

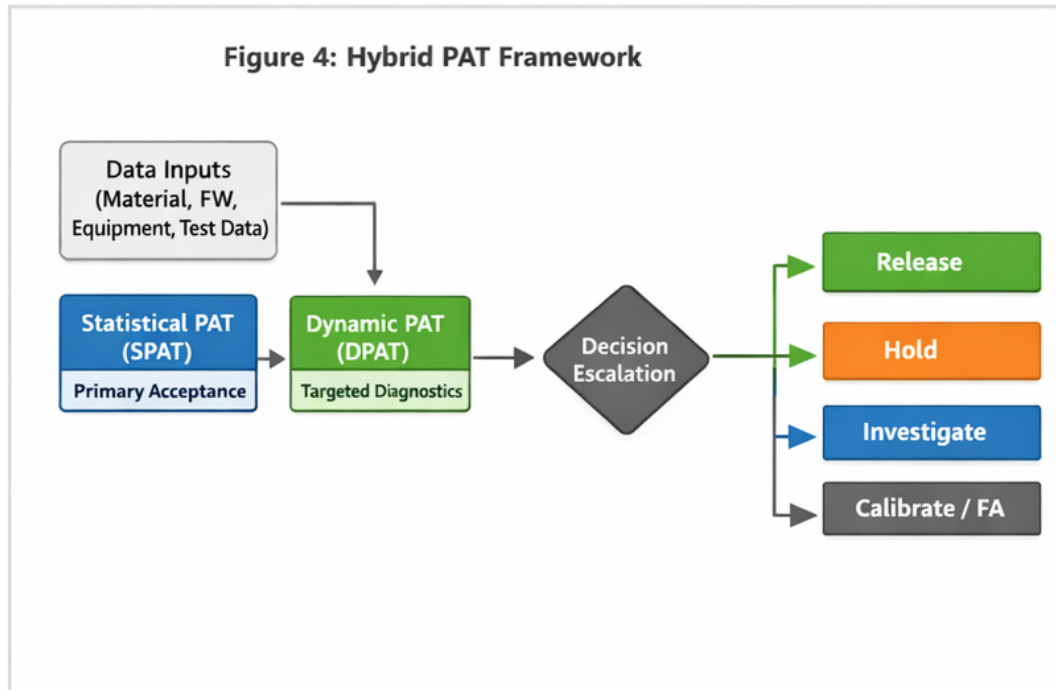


Figure 4: Hybrid framework: SPAT gate + DPAT targeted diagnostics.

9.2. Observed issue

DPAT flagged abnormal drift in a key parametric metric (Param_X) and triggered multiple lot holds.

9.3. Investigation findings

Key contributors:

- Firmware revision update changed calibration/training behavior.
- Socket/contact wear introduced gradual measurement bias.
- Handler thermal drift amplified non-linear behavior.

9.4. Root cause

The DPAT rolling baseline treated the latest population as “equivalent” although the underlying system state changed. This destabilized DPAT thresholds and increased false reject probability.

9.5. SPAT outcome

SPAT baseline remained stable and correctly categorized the effect as a controlled shift, prompting targeted tool qualification actions instead of broad product holds.

10. Simulation Study: Rolling Baseline Fragility in DPAT

10.1. Objective

To quantify DPAT vs SPAT performance under stacked variability, using a simulated backend system with:

- material source mix (A/B),
- firmware version mix (v1/v2),
- multiple testers (T1/T2/T3),
- socket drift over time,
- GR&R measurement noise.

10.2. Method

We define a synthetic parametric metric X with a base distribution plus additive shifts:

$$X = X_{base} + \Delta_{mat} + \Delta_{fw} + \Delta_{tester} + \Delta_{wear}(t) + \epsilon_{GRR}$$

where ϵ_{GRR} represents measurement system noise.

10.3. Evaluation metrics

- False Reject Rate (FRR)
- False Accept Rate (FAR)
- Detection lead time under real drift
- Threshold stability index (baseline variance)

10.4. Expected outcome

DPAT shows faster detection under stable conditions but degrades significantly when untracked variability and baseline drift exist. SPAT remains stable under high-mix conditions.

11. Simulation Study: Multi-Factor Variability Impact on PAT for Read Current

11.1. Motivation

A common misconception in backend test is that a single parametric metric (e.g., Read Current) reflects only the device behavior. In practical backend manufacturing, **Read Current is a multi-factor system output** influenced by product configuration, firmware behavior, test condition variation, and measurement chain effects. As a result, Part Average Testing (PAT) thresholds can become unstable, especially when computed using dynamic rolling populations (DPAT).

This simulation study models how stacked variability sources can cause:

- gradual noise accumulation and baseline drift,
- periods of apparent stability,
- sudden mean shifts caused by discrete changes (e.g., firmware mode mix changes or contact degradation step changes),
- increased false rejects/false accepts in DPAT compared to SPAT.

11.2. Parameter of interest: Read Current

Let the key monitored metric be Read Current, denoted as I_{read} . Although measured as a single scalar value at test, I_{read} is affected by:

1. **Material influence:** material stack and component mix affecting effective resistance, leakage, and timing margin behavior.
2. **Firmware speed mode:** high-speed mode requires higher switching activity and performance margin, leading to increased current consumption.
3. **Tester/thermal condition:** higher testing temperature increases leakage and can raise current consumption.
4. **Measurement chain effects:** degraded contacts, corrosion, or maintenance-related tester leakage introduce additional bias and noise.

11.3. Model formulation

We model I_{read} as:

$$I_{\text{read}}(t) = I_0 + \Delta_{\text{mat}} + \Delta_{\text{fw}} + \Delta_T(T) + \Delta_{\text{hw}}(t) + \epsilon(t)$$

Where:

- I_0 : nominal baseline read current
- Δ_{mat} : material-dependent shift (supplier/lot mix)
- Δ_{fw} : firmware mode-dependent shift (fast vs slow operation)
- $\Delta_T(T)$: temperature effect term (thermal leakage + dynamic current impact)
- $\Delta_{\text{hw}}(t)$: tester HW and contact condition term (slow drift + potential step shifts)
- $\epsilon(t)$: measurement noise (GR&R + local random error)

11.4. Key assumptions

- Firmware speed modes are discretized into:
 - **Fast mode** (higher performance): higher I_{read}
 - **Slow mode** (lower performance): lower I_{read}
- Temperature variation is modeled as a continuous variable, with monotonic impact on current.
- Tester hardware condition is modeled as a combination of:
 - gradual drift (contact wear / increasing leakage),
 - occasional sudden step changes (maintenance events, contact failure, or cleaning-driven erosion).

11.5. Relevance to SPAT vs DPAT

In DPAT, thresholds are computed from recent data windows. When I_{read} contains both gradual drift and discrete shifts, dynamic rolling thresholds tend to:

- **track noise instead of signal** (limit instability),
- **amplify false rejects** during drift,
- **mask escapes** if the rolling window baseline itself degrades.

In SPAT, thresholds derived from long-window historical populations remain stable and robust to short-term variability, making SPAT more suitable as the primary acceptance gate in high-mix backend assembly and test.

12. Interactive Simulation Extension (Portfolio)

To enable engineering teams and decision-makers to explore sensitivity interactively, an optional interactive simulation environment can be provided via portfolio web application, allowing users to:

- adjust variability sliders (FW, materials, tester mix, GR&R, wear drift),
- observe distribution changes and dynamic threshold movement,
- compare SPAT vs DPAT screening behavior.

Note: The interactive simulator uses synthetic data and does not represent any proprietary process limits.

13. Conclusion

Backend assembly and test systems naturally introduce stacked variability and incomplete traceability. In such environments, DPAT's rolling baselines can become unstable and increase false screening decisions. SPAT provides a more robust primary screening mechanism. DPAT becomes most effective when backend stability and traceability maturity are high. A hybrid approach offers the best practical balance: SPAT gate plus DPAT diagnostics.

Appendix: Figure List

- Figure 1: Variance Stack in Backend Assembly & Test
- Figure 2: DPAT Dependency Model
- Figure 3: SPAT vs DPAT Comparison Table
- Figure 4: Hybrid PAT Framework